

Prompt: Fix Crash Planning + Issues Detectados

CRASH PRINCIPAL — Planning queda en blanco

Diagnóstico

El problema NO es el código del planning en sí. La causa es una **incompatibilidad de versiones en** `package.json`:

```
"@dnd-kit/core": "^6.3.1",  
"@dnd-kit/sortable": "^10.0.0",    ← INCOMPATIBLE
```

`@dnd-kit/sortable` versión 10.x requiere `@dnd-kit/core` versión 7.x o superior. Cuando Replit instala las dependencias, el peer dependency check falla o instala versiones incompatibles, lo que hace que el módulo de planning explote en runtime con un error de módulo que deja la pantalla en blanco.

Importante: `@dnd-kit/sortable` no se usa en ningún lugar del código (no hay ningún `import` de él). Está en `package.json` pero nunca se importa.

Fix en `package.json`

Opción A (recomendada) — eliminar sortable que no se usa:

```
"dependencies": {  
  "@dnd-kit/core": "^6.3.1",  
  "@dnd-kit/utilities": "^3.2.2",  
  // eliminar esta línea: "@dnd-kit/sortable": "^10.0.0",  
}
```

Opción B — alinear versiones:

```
"@dnd-kit/core": "^6.3.1",  
"@dnd-kit/sortable": "^7.0.0",    ← versión compatible con core 6.x
```

Después de cambiar el `package.json`, correr en Replit:

```
npm install
```

Bug crítico — `updateReservation` sigue sin el fix de campos seguros

Diagnóstico

`storage.updateReservation` todavía hace:

```
db.update(reservations).set(reservation as any).where(...)
```

Esto pisa campos con null/vacío cuando el frontend manda el objeto completo. Este es el bug que causa que editar una reserva la “borre” visualmente.

Fix en `server/storage.ts`

Buscar `async updateReservation` y reemplazar:

```
async updateReservation(id: string, reservation: Partial<InsertReservation>): Promise<boolean> {
  const safeData: Record<string, any> = {};
  const protectedFields = ["id", "createdAt", "reservationCode"];
  const fkFields = ["guestId", "roomId", "roomTypeId"];

  for (const [key, value] of Object.entries(reservation)) {
    if (protectedFields.includes(key)) continue;
    if (value === undefined) continue;
    if (fkFields.includes(key) && value === "") continue;
    safeData[key] = value;
  }

  if (Object.keys(safeData).length === 0) return undefined;

  const [updated] = await db.update(reservations)
    .set(safeData)
    .where(eq(reservations.id, id))
    .returning();
  return updated;
}
```

Bug — Pago en checkout no avanza (pagos no se reflejan en el modal)

Diagnóstico

`enrichReservation` incluye `charges` pero **no incluye** `payments` en el objeto retornado. Entonces cuando el `ReservationDetailModal` hace:

```
reservation.payments?.reduce((s, p) => s + parseFloat(p.amount), 0) || 0
```

Siempre es `0` aunque haya pagos registrados. El total del saldo nunca baja y el usuario no ve que el pago se registró.

Además, al invalidar `queryKey: ["/api/reservations", reservationId]` después del pago, el re-fetch trae la reserva sin pagos (porque `enrichReservation` no los incluye), entonces el balance sigue mostrando el monto original.

Fix en `server/storage.ts` — agregar `payments` a `enrichReservation`

```
private async enrichReservation(reservation: Reservation): Promise<ReservationWith> {
  const [guest] = await db.select().from(guests).where(eq(guests.id, reservation.guestId));
  const [room] = await db.select().from(rooms).where(eq(rooms.id, reservation.roomId));
  const roomType = room ? (await db.select().from(roomTypes).where(eq(roomTypes.id, room.typeId)).first()) : undefined;
  const ratePlan = reservation.ratePlanId ? (await db.select().from(ratePlans).where(eq(ratePlans.id, reservation.ratePlanId)).first()) : undefined;
  const chargesList = await db.select().from(charges).where(eq(charges.reservationId, reservation.id));
  const paymentsList = await db.select().from(payments).where(eq(payments.reservationId, reservation.id));
  const company = reservation.companyId ? (await db.select().from(companies).where(eq(companies.id, reservation.companyId)).first()) : undefined;
  const agency = reservation.agencyId ? (await db.select().from(agency).where(eq(agency.id, reservation.agencyId)).first()) : undefined;

  return {
    ...reservation,
    guest: guest!,
    company,
    agency,
    room: room ? { ...room, roomType } : undefined as any,
    ratePlan,
    charges: chargesList,
    payments: paymentsList, // ← agregar
  };
}
```

Issue — `earlyCheckIn` y `lateCheckOut` comparación incorrecta

Diagnóstico

En `startEditing()` dentro de `ReservationDetailModal` :

```
setEditEarlyCheckIn(reservation.earlyCheckIn === "true" || reservation.earlyCheck
```

El campo viene de la DB como boolean pero se compara con string `"true"` . Esto puede causar que el early/late no se refleje correctamente al editar.

Fix

```
setEditEarlyCheckIn(!reservation.earlyCheckIn);  
setEditLateCheckOut(!reservation.lateCheckOut);
```

Issue — @dnd-kit/sortable ^10.0.0 en package.json no se usa

Ya cubierto en el Fix del crash arriba. Simplemente eliminar esa línea de `package.json` .

Issue — Botón “Hoy” usa UTC en lugar de hora argentina

Diagnóstico

La función `getLocalToday()` ya está correcta (usa `America/Argentina/Buenos_Aires`), pero en algunos lugares del código todavía se usa:

```
new Date().toISOString().split("T")[0] // ← UTC, no Argentina
```

Fix — reemplazar en todos los lugares del planning y reservas

Buscar todas las ocurrencias de `.toISOString().split("T")[0]` y reemplazar con `getLocalToday()` o `getArgentinaToday()` según el contexto (frontend vs backend).

En **frontend** usar `getLocalToday()` :

```
function getLocalToday() {  
  return new Date().toLocaleDateString("en-CA", { timeZone: "America/Argentina/Bu  
}
```

En **backend** usar `getArgentinaToday()` (ya definida).

Issue — `updateGroup` y `updateCompany` tienen el mismo problema que `updateReservation`

Diagnóstico

Mismo bug: `.set(group as any)` sin filtrar campos vacíos.

Fix en `server/storage.ts`

```
async updateGroup(id: string, group: Partial<InsertGroup>): Promise<Group | undef
  const safeData: Record<string, any> = {};
  for (const [key, value] of Object.entries(group)) {
    if ([ "id", "createdAt", "groupCode" ].includes(key) || value === undefined) co
    safeData[key] = value;
  }
  if (Object.keys(safeData).length === 0) return undefined;
  const [updated] = await db.update(groups).set(safeData).where(eq(groups.id, id)
  return updated;
}

async updateCompany(id: string, company: Partial<InsertCompany>): Promise<Company
  const safeData: Record<string, any> = {};
  for (const [key, value] of Object.entries(company)) {
    if ([ "id", "createdAt" ].includes(key) || value === undefined) continue;
    safeData[key] = value;
  }
  if (Object.keys(safeData).length === 0) return undefined;
  const [updated] = await db.update(companies).set(safeData).where(eq(companies.i
  return updated;
}

// Aplicar el mismo patrón a updateAgency si existe
async updateAgency(id: string, agency: Partial<InsertAgency>): Promise<Agency | u
  const safeData: Record<string, any> = {};
  for (const [key, value] of Object.entries(agency)) {
    if ([ "id", "createdAt" ].includes(key) || value === undefined) continue;
    safeData[key] = value;
  }
  if (Object.keys(safeData).length === 0) return undefined;
  const [updated] = await db.update(agencies).set(safeData).where(eq(agencies.id,
  return updated;
}
```

✓ Lo que está bien en el código nuevo

- `getLocalToday()` y `getArgentinaToday()` implementadas correctamente con `timezone Argentina`
 - `findReservationForRoomAndDay()` implementada correctamente para abrir la reserva correcta en planning
 - `isCheckInDateValid` y `isCheckOutDateValid` usan rango de ± 1 día (más flexible)
 - `canCheckIn` y `canCheckOut` validan correctamente las fechas
 - Drag & drop de reservas entre habitaciones bien implementado con validación de conflictos
 - `enrichReservation` incluye `agency` correctamente
 - `getReservationsForCheckIn()` ya filtra por fecha (hoy y mañana)
 - `ReservationDetailModal` recibe `prop onEdit` y abre `ReservationFormDialog` completo
 - Módulo de Agencias implementado con `AgencySelector`, página `/agencies`, `commissionRate`
 - Ordenamiento numérico de habitaciones por piso implementado
 - `refetchInterval: 30000` en query de planning
 - Early check-in y late check-out muestran iconos (Sunrise/Sunset) en la grilla
 - `unassignedGroupBlocks` incluidos en `getPlanningData`
 - Fix de `agencyId` en `agencyId` en reservations y guests
-

Orden de aplicación recomendado

1. **Primero:** Fix `package.json` + `npm install` → resuelve el crash del planning
2. **Segundo:** Fix `enrichReservation` → agrega payments → resuelve checkout
3. **Tercero:** Fix `updateReservation`, `updateGroup`, `updateCompany` → evita borrado de datos
4. **Cuarto:** Fix `earlyCheckIn` boolean comparison → cosmético pero correcto

